How word2vec Works

A guided tour of the idea and of the original C implementation

Notes on `word2vec.c` and its companion tools

October 9, 2026

Abstract

word2vec turns every word of a text corpus into a short, dense vector of real numbers such that words used in similar contexts end up close together. This document explains *what* word2vec is trying to achieve, *why* the training objective produces useful vectors, and *how* the original C program in this repository (`word2vec.c`) implements it: vocabulary building, subsampling, the sliding context window, the CBOW and skip-gram architectures, negative sampling, hierarchical softmax, the sigmoid lookup table, the learning-rate schedule and lock-free multithreading. Line numbers such as `word2vec.c:450` refer to the file in this repository. Code listings are excerpts; a few explanatory comments have been added.

Contents

1 The goal: meaning as geometry **3**
- 1.1 The problem with words as symbols . . . 3
- 1.2 The distributional hypothesis . . . 3
- 1.3 What word2vec produces . . . 3
- 1.4 The trick: a fake task . . . 3

2 The big picture: from text file to vectors **3**

3 Step 1: building the vocabulary **4**
- 3.1 Tokenising . . . 4
- 3.2 Counting with a hash table . . . 5
- 3.3 Sorting and pruning . . . 5

4 The model: two matrices and a dot product **5**
- 4.1 The parameters . . . 5
- 4.2 Initialisation . . . 5
- 4.3 The score . . . 6

5 Step 4a: generating training examples **6**
- 5.1 Subsampling frequent words . . . 6
- 5.2 The sliding context window . . . 7

6 Step 4b: the two architectures **7**
- 6.1 CBOW: continuous bag of words . . . 8
- 6.2 Skip-gram . . . 8

7 Step 4c: making the output layer cheap **9**
- 7.1 Negative sampling (`-negative k`) . . . 9
- 7.2 Hierarchical softmax (`-hs 1`) . . . 10

8 Implementation tricks for speed **11**
8.1 The sigmoid lookup table . . . 11
8.2 Learning-rate schedule . . . 12
8.3 Lock-free multithreading (Hogwild) . . . 12

9 Putting it together: one training step **12**

10 Step 5: the output and how it is used **13**
10.1 The vector file . . . 13
10.2 Querying the vectors . . . 13
10.3 Phrases: `word2phrase` . . . 14

11 Hyperparameters at a glance **14**

Appendix: code map **15**

Further reading **15**

1 The goal: meaning as geometry

1.1 The problem with words as symbols

A computer sees a word as an arbitrary symbol. The simplest numerical encoding is a *one-hot* vector: with a vocabulary of V words, word i is a vector of length V with a single 1 at position i. One-hot vectors carry no meaning: every pair of distinct words is equally far apart, so *cat* is as different from *dog* as it is from *carburettor*.

1.2 The distributional hypothesis

word2vec rests on an old idea from linguistics:

> *"You shall know a word by the company it keeps."* — J. R. Firth, 1957

Words that appear in similar contexts tend to have similar meanings. *Cat* and *dog* both appear near *pet*, *fed*, *vet* and *fur*; *carburettor* does not.

1.3 What word2vec produces

word2vec learns a mapping

$$\text{word } w \longmapsto \mathbf{v}_w \in \mathbb{R}^d, \qquad d \approx 100\text{–}300 \ll V,$$

chosen so that words with similar contexts get vectors pointing in similar directions. Figure 1 contrasts the two representations.

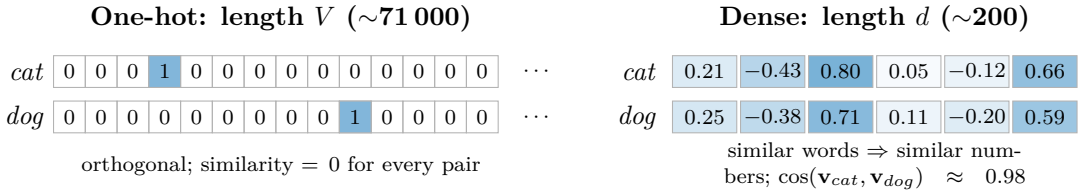


Figure 1: One-hot vectors (left) carry no notion of similarity. word2vec's dense vectors (right) put related words close together. The numbers are illustrative.

The surprising bonus is that the geometry of the space captures *relations*, not only similarity. Differences between vectors encode consistent directions, so simple vector arithmetic solves analogies:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$

1.4 The trick: a fake task

Nobody tells word2vec what words mean. Instead it trains a tiny neural network on a *fake prediction task* that can be generated automatically from raw text: *given a word, predict the words around it* (or the reverse). The prediction itself is thrown away. The useful product is the network's first weight matrix, whose rows are the word vectors. To predict contexts well, the network must give words with similar contexts similar vectors, which is exactly what we wanted.

2 The big picture: from text file to vectors

The repository contains five programs. `word2vec` does the training; the others prepare input or use the output (Figure 3).

`TrainModel()` is short enough to read in full; it is the outline of the whole program:

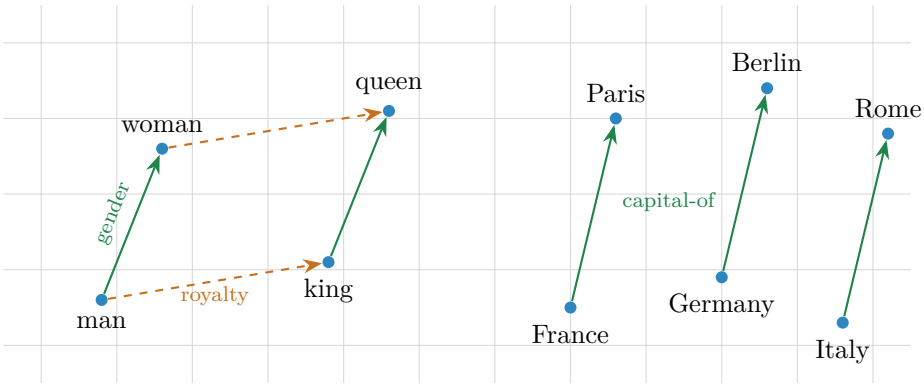


Figure 2: A 2-D cartoon of the learned space. Consistent relationships become roughly parallel offsets, which is why `word-analogy` can answer "*man* is to *king* as *woman* is to __" by computing $\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman}$.

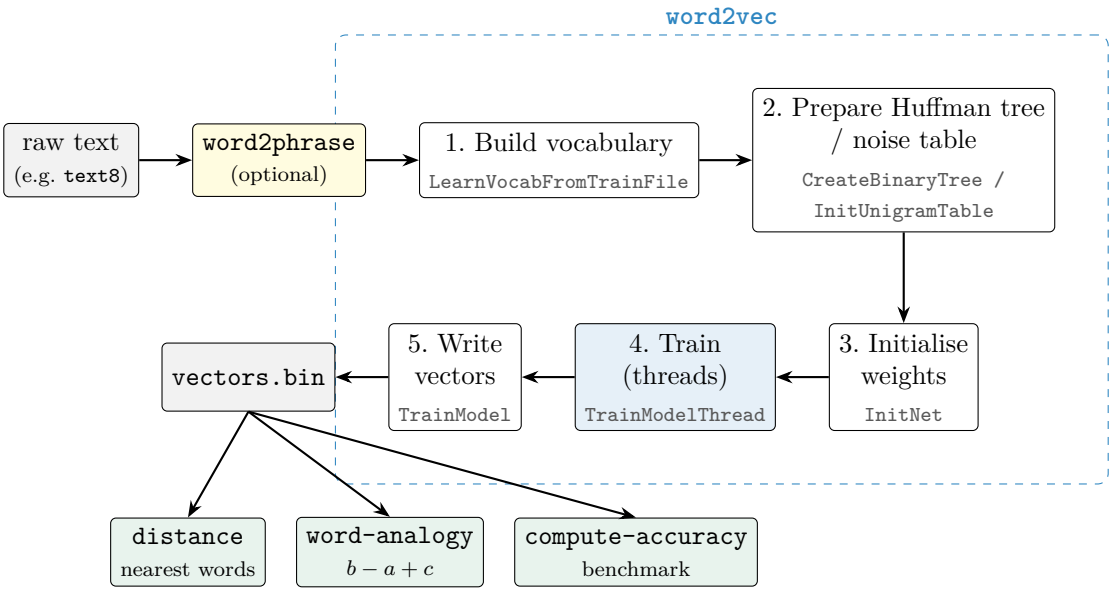


Figure 3: Overall data flow. Everything inside the dashed box happens in `word2vec.c`, driven by `TrainModel()` (`word2vec.c:572`).

```
579   starting_alpha = alpha;
580   if (read_vocab_file[0] != 0) ReadVocab(); else LearnVocabFromTrainFile();
581   if (save_vocab_file[0] != 0) SaveVocab();
582   if (output_file[0] == 0) return;
583   InitNet();
584   if (negative > 0) InitUnigramTable();
585   start = clock();
586   for (a = 0; a < num_threads; a++) pthread_create(&pt[a], NULL, TrainModelThread, (void *)a);
587   for (a = 0; a < num_threads; a++) pthread_join(pt[a], NULL);
```

The rest of this document follows these steps in order.

3 Step 1: building the vocabulary

3.1 Tokenising

`ReadWord()` (`word2vec.c:77`) splits the file on spaces, tabs and newlines. There is no other preprocessing: punctuation and case are kept as they are, so the corpus is normally cleaned

beforehand. Every newline is turned into the special token `</s>`, which marks a sentence boundary and always sits at vocabulary index 0.

3.2 Counting with a hash table

`LearnVocabFromTrainFile()` (`word2vec.c:279`) streams through the corpus once, counting every word. Lookups use an open-addressing hash table (`vocab_hash`, 30 million slots) with linear probing:

```
104 int GetWordHash(char *word) {
105   unsigned long long a, hash = 0;
106   for (a = 0; a < strlen(word); a++) hash = hash * 257 + word[a];
107   hash = hash % vocab_hash_size;
108   return hash;
109 }
```

If the table becomes 70% full, `ReduceVocab()` (`word2vec.c:191`) throws away all words seen at most `min_reduce` times and increments `min_reduce`, so huge corpora still fit in memory.

3.3 Sorting and pruning

`SortVocab()` (`word2vec.c:161`) sorts words by descending count (keeping `</s>` first) and discards words seen fewer than `-min-count` times (default 5). Rare words have too few contexts to learn a good vector, and dropping them shrinks the model a lot. After sorting, *a word's index is its frequency rank*; several later steps rely on this.

Each entry is a `struct vocab_word`:

```
37 struct vocab_word {
38   long long cn;          // count in the corpus
39   int *point;            // Huffman path: inner-node indices (hierarchical softmax)
40   char *word, *code, codelen;  // the string; Huffman code bits and their number
41 };
```

4 The model: two matrices and a dot product

4.1 The parameters

Despite the "neural network" label, word2vec's model is simply two $V \times d$ matrices of floats, where d is `layer1_size` (`-size`):

Array	Symbol	Role
`syn0`	$\mathbf{v}_w$	**input vectors**: the word vectors written to the output file
`syn1neg`	$\mathbf{u}_w$	output vectors used by negative sampling
`syn1`	$\mathbf{u}_n$	output vectors of the inner nodes of the Huffman tree (hierarchical softmax)

All are stored as flat row-major arrays, so the vector of word w starts at offset `w * layer1_size`. That is why the training loop is full of `l1 = last_word * layer1_size` and `l2 = target * layer1_size`.

4.2 Initialisation

`InitNet()` (`word2vec.c:361`) fills `syn0` with small random numbers, uniform in $[-\frac{0.5}{d}, \frac{0.5}{d}]$, and sets the output matrices to zero:

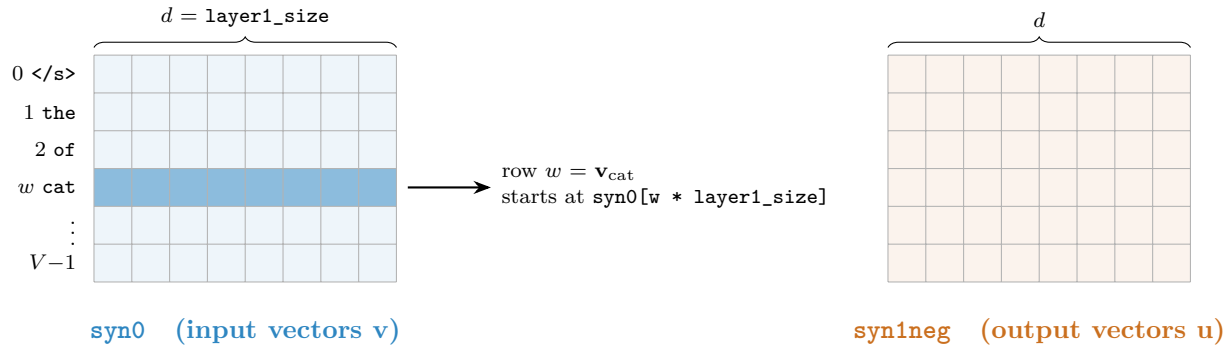


Figure 4: Memory layout of the two weight matrices. Every word has *two* vectors: one used when it is the input, one used when it is the prediction target. Only `syn0` is saved.

```
378   for (a = 0; a < vocab_size; a++) for (b = 0; b < layer1_size; b++) {
379     next_random = next_random * (unsigned long long)25214903917 + 11;
380     syn0[a * layer1_size + b] = (((next_random & 0xFFFF) / (real)65536) - 0.5) / layer1_size;
381   }
```

The expression `next_random * 25214903917 + 11` is a linear congruential random-number generator. It appears throughout the code because it is much faster than calling `rand()`.

4.3 The score

The model's only computation is a dot product. The score for "word o appears in the context of word c" is

$$s(c, o) = \mathbf{v}_c^\top \mathbf{u}_o,$$

turned into a probability with the logistic sigmoid $\sigma(x) = 1/(1 + e^{-x})$. High dot product means "likely neighbours".

5 Step 4a: generating training examples

Each training thread runs `TrainModelThread()` (`word2vec.c:385`). Its outer loop reads one "sentence" at a time (up to the next `</s>`, at most 1000 words) into the array `sen[]`, then slides over it word by word.

5.1 Subsampling frequent words

Very frequent words such as *the*, *of* and *a* appear in nearly every context and so carry little information. They also dominate training time. While reading a sentence, the code randomly drops each occurrence of word w:

```
423         if (sample > 0) {
424           real ran = (sqrt(vocab[word].cn / (sample * train_words)) + 1) * (sample *
      train_words) / vocab[word].cn;
425           next_random = next_random * (unsigned long long)25214903917 + 11;
426           if (ran < (next_random & 0xFFFF) / (real)65536) continue;
427         }
```

Writing $f(w) = \texttt{cn}/\texttt{train_words}$ for the word's relative frequency and t for `-sample`, the probability of *keeping* an occurrence is

$$P_{\text{keep}}(w) = \left(\sqrt{\frac{f(w)}{t}} + 1 \right) \frac{t}{f(w)}.$$

Words with $f(w)$ below about $2.6\,t$ have $P_{\text{keep}} \geq 1$ and are always kept; frequent words are aggressively thinned (Figure 5). Dropping words also effectively *widens* the context window, since the surviving neighbours were originally further apart.

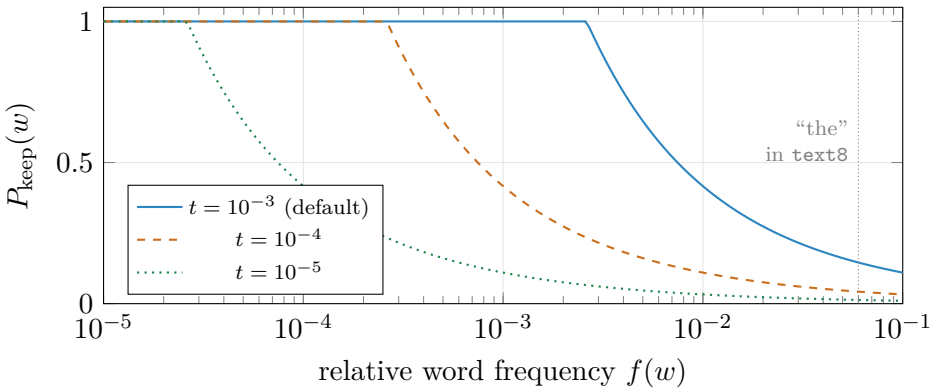


Figure 5: Probability that an occurrence survives subsampling. Rare words are always kept; with $t = 10^{-4}$ only about 4% of the occurrences of *the* survive.

5.2 The sliding context window

For every position in the sentence, the current word `word` is the *centre*. Its context is the words up to `window` positions to the left and right. The code first draws a random shrink amount `b` in $[0, \texttt{window} - 1]$, so the effective window size is $\texttt{window} - b$, uniformly between 1 and `window`:

```
448     next_random = next_random * (unsigned long long)25214903917 + 11;
449     b = next_random % window;
```

```
453       for (a = b; a < window * 2 + 1 - b; a++) if (a != window) {
454         c = sentence_position - window + a;   // index of a context word
```

This is a cheap way to give near neighbours more weight: a word at distance 1 is always inside the window, a word at distance `window` only $1/\texttt{window}$ of the time.

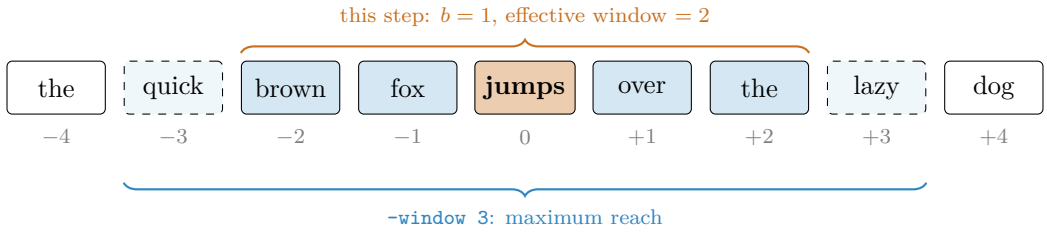


Figure 6: The context window around the centre word *jumps* (at `sentence_position`). Solid blue words are used in this step; dashed ones are inside the maximum window but cut by the random shrink b.

6 Step 4b: the two architectures

The `-cbow` flag chooses how the centre word and its context are turned into predictions (Figure 7).

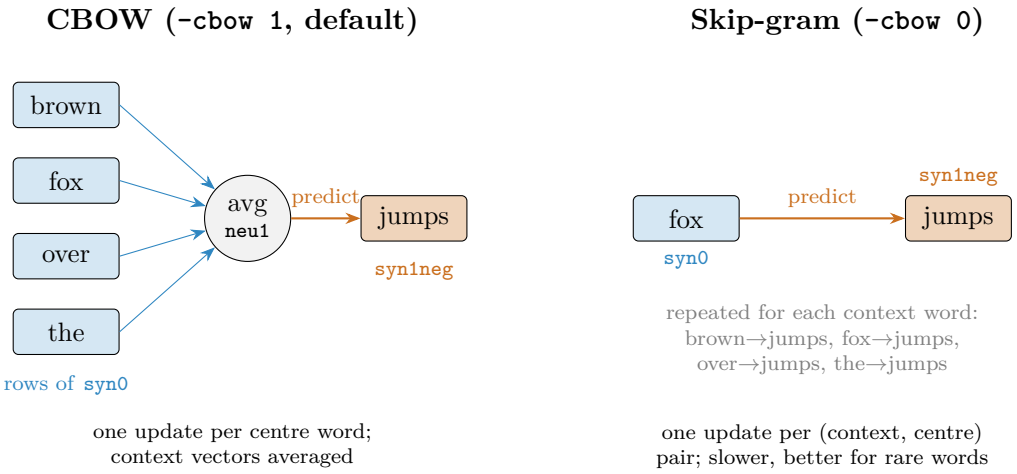


Figure 7: The two architectures for the window of Figure 6. Blue boxes are looked up in `syn0`; the orange box is the prediction target, scored with `syn1neg` (or `syn1`).

6.1 CBOW: continuous bag of words

CBOW predicts the centre word from its context. The context vectors are *averaged* into a hidden vector $\mathbf{h}$ (`neu1`). Word order inside the window is ignored, hence "bag of words":

$$\mathbf{h} = \frac{1}{|C|} \sum_{c \in C} \mathbf{v}_c.$$

```
459         for (c = 0; c < layer1_size; c++) neu1[c] += syn0[c + last_word * layer1_size];
460         cw++;
461       }
462       if (cw) {
463         for (c = 0; c < layer1_size; c++) neu1[c] /= cw;
```

$\mathbf{h}$ is then scored against the centre word's output vector. The resulting error vector `neu1e` is added, *undivided*, to every context word's input vector (`word2vec.c:501-508`).

6.2 Skip-gram

Skip-gram does one prediction per (context word, centre word) pair. The paper describes it as "predict each context word from the centre word". The code does the mirror image: it uses the *context* word's input vector (`l1 = last_word * layer1_size`) to predict the *centre* word (`word`) at the output. Because every pair is visited both ways as the window slides, the two formulations see the same set of pairs.

```
511       for (a = b; a < window * 2 + 1 - b; a++) if (a != window) {
512         ...
513         last_word = sen[c];
514         l1 = last_word * layer1_size;   // input: context word
515         for (c = 0; c < layer1_size; c++) neu1e[c] = 0;
516         // ... score syn0[l1] against the centre word 'word' ...
517         for (c = 0; c < layer1_size; c++) syn0[c + l1] += neu1e[c];
518       }
```

In both cases the remaining question is: *how do we turn "this input should predict that word" into a gradient cheaply?*

7 Step 4c: making the output layer cheap

The textbook objective is a softmax over the whole vocabulary:

$$p(o \mid c) = \frac{\exp(\mathbf{v}_c^\top \mathbf{u}_o)}{\sum_{w=1}^{V} \exp(\mathbf{v}_c^\top \mathbf{u}_w)}.$$

The denominator touches all V output vectors, roughly 71 000 dot products for `text8` and millions for large corpora, *for every single training pair*. word2vec offers two approximations that cost $O(k)$ or $O(\log V)$ instead. They can be used together, but normally one is chosen: `-negative 5 -hs 0` (the default) or `-negative 0 -hs 1`.

7.1 Negative sampling (`-negative k`)

Negative sampling replaces the V-way classification with $k+1$ independent yes/no questions: "is this pair a real (context, word) pair from the corpus, or noise?" The real target is the one positive example; k words drawn at random are negatives (Figure 8). For input vector $\mathbf{h}$ (CBOW average or skip-gram context vector) and true word o, the objective to *maximise* is

$$\mathcal{L} = \log \sigma\left(\mathbf{h}^\top \mathbf{u}_o\right) + \sum_{i=1}^{k} \log \sigma\left(-\mathbf{h}^\top \mathbf{u}_{n_i}\right), \qquad n_i \sim P_n(w).$$

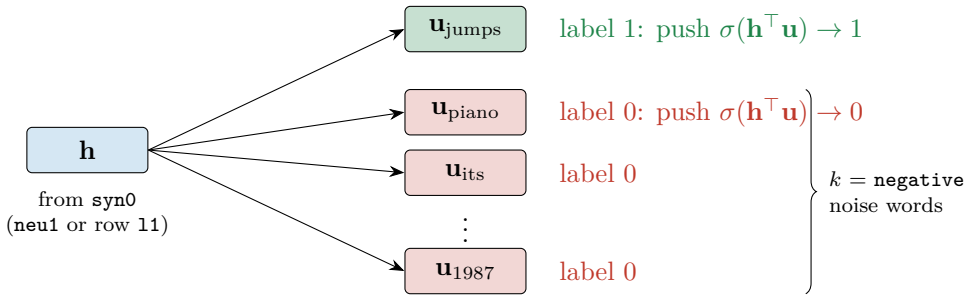


Figure 8: Negative sampling: one positive (the real centre word) and k random negatives. Only these $k+1$ rows of `syn1neg` are touched.

The gradient. For one term with label $\ell \in \{0, 1\}$ and score $x = \mathbf{h}^\top \mathbf{u}$, the derivative of $\ell \log \sigma(x) + (1-\ell) \log \sigma(-x)$ with respect to x is simply

$$\frac{\partial \mathcal{L}}{\partial x} = \ell - \sigma(x).$$

This is the whole learning rule. The code calls the step size times this error $\mathbf{g}$, then applies the chain rule to both vectors involved:

$$g = \alpha\,(\ell - \sigma(x)), \qquad \mathbf{u} \leftarrow \mathbf{u} + g\,\mathbf{h}, \qquad \mathbf{e} \leftarrow \mathbf{e} + g\,\mathbf{u}.$$

$\mathbf{e}$ (`neu1e`) accumulates the error for the input side. It is applied to `syn0` only after all $k+1$ terms are processed, so every term sees the same $\mathbf{h}$. Here is the CBOW version:

```
480         if (negative > 0) for (d = 0; d < negative + 1; d++) {
481           if (d == 0) {
482             target = word;              // the real centre word
483             label = 1;
484           } else {
485             next_random = next_random * (unsigned long long)25214903917 + 11;
486             target = table[(next_random >> 16) % table_size];   // noise word
```

```
487             if (target == 0) target = next_random % (vocab_size - 1) + 1;
488             if (target == word) continue;
489             label = 0;
490           }
491           l2 = target * layer1_size;
492           f = 0;
493           for (c = 0; c < layer1_size; c++) f += neu1[c] * syn1neg[c + l2];  // x = h . u
494           if (f > MAX_EXP) g = (label - 1) * alpha;
495           else if (f < -MAX_EXP) g = (label - 0) * alpha;
496           else g = (label - expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))]) *
      alpha;
497           for (c = 0; c < layer1_size; c++) neu1e[c] += g * syn1neg[c + l2];  // e += g u
498           for (c = 0; c < layer1_size; c++) syn1neg[c + l2] += g * neu1[c];   // u += g h
499         }
```

Intuitively: if a real neighbour scores low, $g > 0$ pulls $\mathbf{h}$ and $\mathbf{u}_o$ together; if a noise word scores high, $g < 0$ pushes them apart. Over millions of updates, words that share neighbours get pulled towards the same output vectors and so towards each other.

The noise distribution. Negatives are drawn from the unigram distribution raised to the power 3/4:

$$P_n(w) = \frac{\texttt{cn}(w)^{0.75}}{\sum_{w'} \texttt{cn}(w')^{0.75}}.$$

The exponent flattens the distribution, so rare words are picked more often than their raw frequency and common words less. To sample in $O(1)$, `InitUnigramTable()` (`word2vec.c:58`) fills a table of 10^8 integers where word w occupies a share of slots proportional to $P_n(w)$; drawing a uniformly random slot then samples from P_n (Figure 9).

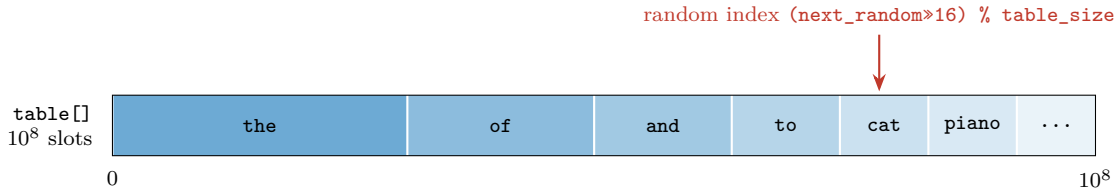


Figure 9: The unigram table. Each word owns a contiguous run of slots, sized by $\texttt{cn}^{0.75}$, so a single random index yields a noise word. Proportions are illustrative.

7.2 Hierarchical softmax (`-hs 1`)

Hierarchical softmax keeps a true probability distribution over all words but computes it as a sequence of binary decisions along a path in a binary tree whose leaves are the words. The probability of a word is the product of the probabilities of taking each left/right turn on the way from the root to its leaf. Each inner node n has its own vector $\mathbf{u}_n$ (a row of `syn1`), and the probability of turning one way at that node is $\sigma(\mathbf{h}^\top \mathbf{u}_n)$. Only $O(\log V)$ nodes lie on a path.

The Huffman tree. `CreateBinaryTree()` (`word2vec.c:214`) builds a *Huffman* tree from the word counts, so frequent words get short paths and are cheap to train. Because the vocabulary is already sorted by count, the classic algorithm runs in linear time with two pointers: `pos1` walks left over the leaves (smallest first) and `pos2` walks right over the newly created inner nodes. For each word the code stores

- `code[]`: the bits (0/1) of the turns from root to leaf, and
- `point[]`: the indices of the inner nodes visited, i.e. which rows of `syn1` to use.

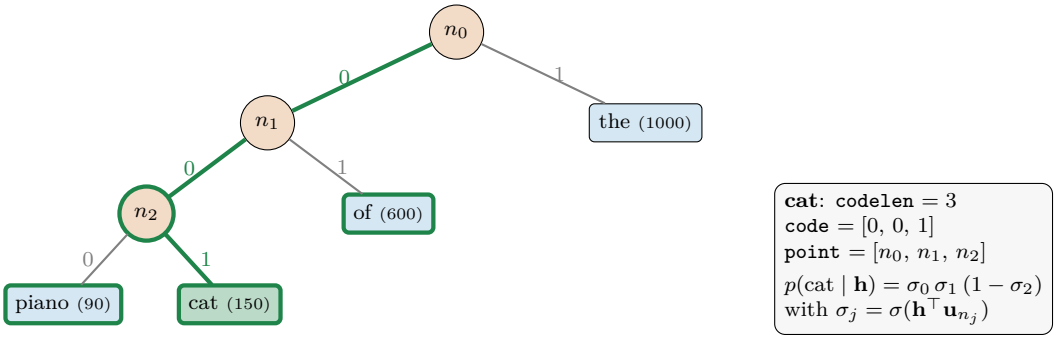


Figure 10: A toy Huffman tree built as `CreateBinaryTree()` does it: repeatedly merge the two smallest nodes, giving bit 0 to the smaller and 1 to the larger (`binary[min2i] = 1`). The most frequent word sits closest to the root. To predict *cat*, the model only scores the three inner nodes on the highlighted path instead of all V words.

The update. At each node on the path the code does one logistic-regression step whose label is 1 − code bit. A 0 bit (turn "left") is the positive class:

```
464         if (hs) for (d = 0; d < vocab[word].codelen; d++) {
465           f = 0;
466           l2 = vocab[word].point[d] * layer1_size;   // inner node on the path
467           for (c = 0; c < layer1_size; c++) f += neu1[c] * syn1[c + l2];
468           if (f <= -MAX_EXP) continue;
469           else if (f >= MAX_EXP) continue;
470           else f = expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))];
471           g = (1 - vocab[word].code[d] - f) * alpha;  // (label - sigma) * alpha
472           for (c = 0; c < layer1_size; c++) neu1e[c] += g * syn1[c + l2];
473           for (c = 0; c < layer1_size; c++) syn1[c + l2] += g * neu1[c];
474         }
```

This is the same rule $g = \alpha(\ell - \sigma(x))$ as in negative sampling. The only difference is where the binary questions come from: tree nodes instead of random words.

Which one to use? Hierarchical softmax tends to do better for infrequent words; negative sampling does better for frequent words and with low-dimensional vectors. The defaults (`-negative 5 -hs 0`) are a good start; the original README suggests skip-gram with negative sampling for quality and CBOW for speed.

8 Implementation tricks for speed

8.1 The sigmoid lookup table

Computing `exp()` for every update would be slow, so `main()` precomputes $\sigma(x)$ at 1000 points on $[-6, 6]$ (`word2vec.c:739-742`):

```
739   for (i = 0; i < EXP_TABLE_SIZE; i++) {
740     expTable[i] = exp((i / (real)EXP_TABLE_SIZE * 2 - 1) * MAX_EXP); // Precompute the exp()
        table
741     expTable[i] = expTable[i] / (expTable[i] + 1);                   // Precompute f(x) = x /
        (x + 1)
742   }
```

At training time a score x is mapped to the index $\lfloor (x + 6) \cdot 83 \rfloor$. Outside $[-6, 6]$ the sigmoid is treated as saturated: negative sampling uses exactly 0 or 1, and hierarchical softmax skips the node. The error is negligible.

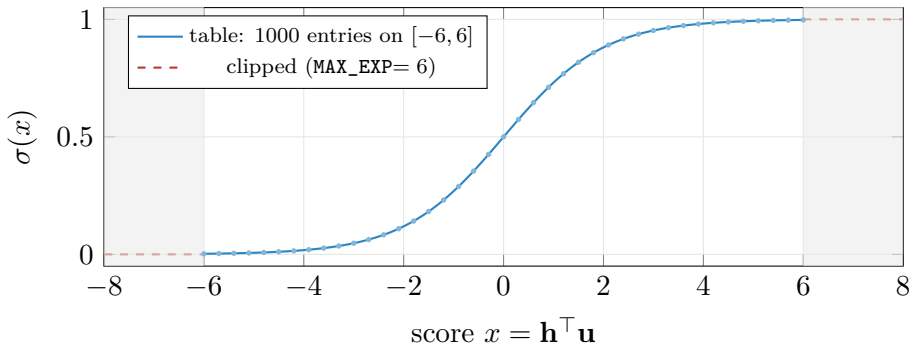


Figure 11: The sigmoid is read from `expTable` inside $[-6, 6]$ and treated as 0 or 1 outside it (shaded).

8.2 Learning-rate schedule

The step size α starts at `-alpha` (0.025 for skip-gram, 0.05 for CBOW) and decreases linearly to almost zero over the whole run. Every 10 000 words, each thread recomputes it from the global progress counter:

```
412        alpha = starting_alpha * (1 - word_count_actual / (real)(iter * train_words + 1));
413        if (alpha < starting_alpha * 0.0001) alpha = starting_alpha * 0.0001;
```

Large early steps move the random vectors quickly into a sensible arrangement; small late steps fine-tune it without jitter.

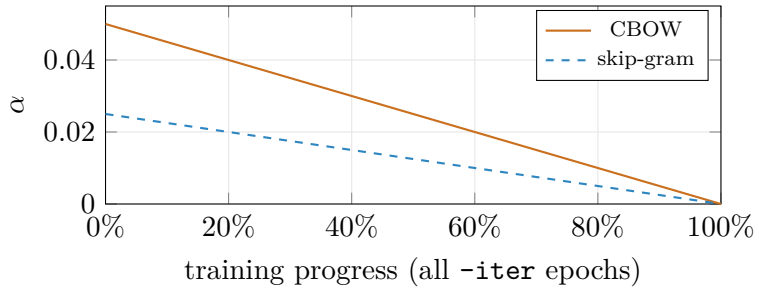


Figure 12: Linear learning-rate decay, floored at $10^{-4}\times$ the starting value.

8.3 Lock-free multithreading (Hogwild)

`TrainModel()` starts `-threads` threads. Thread i seeks to byte offset $i \cdot$ `file_size/num_threads` in the training file and processes `train_words/num_threads` words per epoch, repeated for `-iter` epochs (`word2vec.c:400`, `word2vec.c:434-443`).

All threads read and write the *same* `syn0` and `syn1neg` arrays without any locks. Two threads can occasionally overwrite each other's update to the same row, but each update touches only a handful of rows out of tens of thousands, so collisions are rare and the lost updates act like a little extra noise. This "Hogwild" style of parallel stochastic gradient descent scales almost linearly with the number of cores.

9 Putting it together: one training step

Figure 14 condenses `TrainModelThread()` for skip-gram with negative sampling (the CBOW path differs only in averaging the context first).

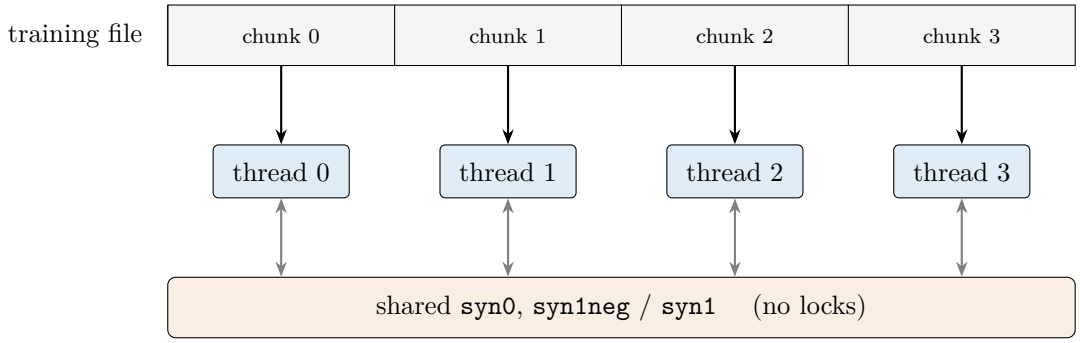


Figure 13: Each thread trains on its own slice of the file, while all threads update the shared weight matrices concurrently.

for each epoch, **for** each sentence in this thread's chunk:
  read words up to `</s>`; drop each occurrence of w with probability $1 - P_{\text{keep}}(w)$
  for each position p with centre word o:
    $b \leftarrow$ random in $[0, \texttt{window})$
    for each context word c within $\texttt{window} - b$ of p:
      $\mathbf{e} \leftarrow \mathbf{0}$
      for (t, ℓ) in $\{(o, 1)\} \cup \{(n_i, 0)\}_{i=1}^{k}$, $\quad n_i \sim P_n$:
        $x \leftarrow \mathbf{v}_c^{\top} \mathbf{u}_t$; $\quad g \leftarrow \alpha(\ell - \sigma(x))$
        $\mathbf{e} \leftarrow \mathbf{e} + g\,\mathbf{u}_t$; $\quad \mathbf{u}_t \leftarrow \mathbf{u}_t + g\,\mathbf{v}_c$
      $\mathbf{v}_c \leftarrow \mathbf{v}_c + \mathbf{e}$
  every 10 000 words: $\alpha \leftarrow \alpha_0 \max\bigl(1 - \text{progress}, 10^{-4}\bigr)$

Figure 14: One thread's training loop (skip-gram, negative sampling).

10 Step 5: the output and how it is used

10.1 The vector file

After all threads finish, `TrainModel()` writes `syn0`, the input vectors. The first line is "V d". Each following line holds a word and its d components, as text or, with `-binary 1`, as raw 4-byte floats (`word2vec.c:593-601`). The output matrices are discarded.

With `-classes K`, the program instead runs 10 iterations of k-means on the vectors (using normalised centroids and dot-product similarity) and writes one "word cluster-id" line per word (`word2vec.c:602-645`).

10.2 Querying the vectors

All query tools load the file, normalise each vector to unit length, and compare vectors by *cosine similarity*:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^{\top}\mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|} \in [-1, 1].$$

Program	What it does
`distance`	Reads one or more words, averages and normalises their vectors, and prints the 40 vocabulary words with the highest cosine similarity (excluding the query words).
`word-analogy`	Reads three words *a b c*, computes $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$, and prints the 40 nearest words, e.g. *paris france berlin* $\rightarrow$ *germany*.
`compute-accuracy`	Runs the analogy questions in `questions-words.txt` ("*athens greece baghdad iraq*"), restricted to the most frequent words (e.g. 30 000), and reports top-1 accuracy split into semantic and syntactic sections.

10.3 Phrases: `word2phrase`

Some meanings belong to multi-word units: *new york*, *air canada*. `word2phrase` makes one pass over the corpus counting unigrams and bigrams, then joins a bigram *ab* into a single token `a_b` when

$$\text{score}(a,b) = \frac{\texttt{cn}(ab) - \texttt{min_count}}{\texttt{cn}(a)\cdot\texttt{cn}(b)} \cdot \texttt{train_words} > \texttt{threshold} \quad \text{(default 100)}.$$

The score is high when a and b occur together far more often than chance would predict. Subtracting `min_count` stops rare coincidences from becoming phrases. Running it twice builds phrases of up to four words, as `demo-phrases.sh` does, and `word2vec` then learns vectors for the phrase tokens like any other word.

11 Hyperparameters at a glance

Option	Default	Effect
`-size`	100	Vector dimension d. Larger captures more but needs more data.
`-window`	5	Maximum context reach. Small windows favour syntactic similarity, large ones topical similarity.
`-cbow`	1	1 = CBOW (fast), 0 = skip-gram (slower, better for rare words).
`-negative`	5	Noise words per positive example; 0 disables negative sampling.
`-hs`	0	1 enables hierarchical softmax.
`-sample`	10^{-3}	Subsampling threshold t; smaller drops more frequent words.
`-alpha`	0.05 / 0.025	Starting learning rate (CBOW / skip-gram).
`-iter`	5	Number of passes over the corpus.
`-min-count`	5	Discard words rarer than this.
`-threads`	12	Parallel training threads.
`-binary`	0	Write vectors as binary floats instead of text.
`-classes`	0	If > 0, output k-means cluster ids instead of vectors.

A typical high-quality run on `text8`, as in the README:

```
./word2vec -train text8 -output vectors.bin -cbow 1 -size 200 -window 8 \
           -negative 25 -hs 0 -sample 1e-4 -threads 8 -binary 1 -iter 15
```

Appendix: code map

Function	Lines	Role
`InitUnigramTable`	58–74	Builds the $\texttt{cn}^{0.75}$ noise table for negative sampling.
`ReadWord`	77–101	Tokeniser; newline becomes `</s>`.
`GetWordHash`, `SearchVocab`	104–120	Hash-table lookup with linear probing.
`AddWordToVocab`	134–150	Inserts a new word, growing the array as needed.
`SortVocab`	161–188	Sort by count, drop rare words, allocate Huffman arrays.
`ReduceVocab`	191–210	Prunes the vocabulary when the hash table fills up.
`CreateBinaryTree`	214–277	Linear-time Huffman tree; fills `code` and `point`.
`LearnVocabFromTrainFile`	279–315	First pass over the corpus: counts words.
`InitNet`	361–383	Allocates and initialises `syn0`, `syn1`, `syn1neg`.
`TrainModelThread`	385–570	The training loop: subsampling, window, CBOW/skip-gram, HS/NS updates.
`TrainModel`	572–647	Orchestrates everything; writes vectors or k-means classes.
`main`	670–745	Parses options, precomputes `expTable`.

Further reading

- T. Mikolov, K. Chen, G. Corrado, J. Dean. *Efficient Estimation of Word Representations in Vector Space*. ICLR Workshop, 2013.
- T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. *Distributed Representations of Words and Phrases and their Compositionality*. NeurIPS, 2013.
- Y. Goldberg, O. Levy. *word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method*. arXiv:1402.3722, 2014.
- `ARCHITECTURE.md` and `BUILD_AND_USAGE.md` in this repository.